

Exercise: Simulating Message-Adversary-Tolerant Byzantine Reliable Broadcast

Davide Frey - davide.frey@inria.fr

Read Albouy, Frey, Raynal, and Taiani, *Asynchronous Byzantine Reliable Broadcast With a Message Adversary* (2022): <https://arxiv.org/abs/2205.09992>.

Goal

Implement Algorithm 1 from the paper and simulate its delivery behavior and fault tolerance. You are modelling the paper's abstraction, not building a production broadcast system.

1. Read and explain the model

Write a short summary of the two adversaries and the MBRB properties. Define n (all processes), t (Byzantine-process bound), d (message-adversary power), and c (correct processes in one run). Explain why the paper assumes:

$$n > 3t + 2d$$

For a correct sender, state the target global-delivery guarantee:

at least $l = c - d$ correct processes deliver

This is not necessarily delivery by every correct process when $d > 0$.

2. Build the shared adversary component

Build the adversaries in your own project. You may use SimPy DS as a dependency or as reference code, and you may fork it, but you do not need to change the original repository.

Groups doing this exercise may work with PBFT groups on this part. Final submissions may share one adversary codebase; list all contributors. The protocol, experiments, and report are still specific to each exercise.

Message adversary

Implement a network extension or a composable message-adversary object. For every implementation-message broadcast by a correct sender, it may suppress copies to at most d correct destinations. Track this budget per logical broadcast, for example by `(sender, broadcast_id)`. Protocol-specific metadata may be included in `broadcast_id`. This is not merely an independent `drop_rate`: the paper bounds how many correct recipients miss each broadcast.

Implement and document at least all of the following deterministic policies for selecting omitted recipients:

- a fixed set of d correct receivers for every broadcast;
- a rotating set selected from a seeded pseudo-random generator; and
- a targeted set, such as receivers that have not yet accumulated a quorum.

You may combine these policies or add others. Every policy must still omit at most d correct recipients per logical broadcast.

Do not allow it to corrupt, duplicate, invent, or modify messages. Its only power is omission of up to d copies sent by a correct process to correct processes.

Byzantine adversary

Represent faulty processes explicitly and let the exercise-specific protocol define correct behavior. Implement at least the first three of the following behaviors. Record which behavior each experiment uses:

- **crash/silence:** send nothing after a chosen time;
- **omission:** receive messages but never relay or sign;
- **equivocation:** sign or authenticate conflicting values for the same logical protocol instance and send them to different peers;
- **selective forwarding:** relay valid bundles to only a chosen subset; and
- **malformed or replayed message:** send invalid authentication, old messages, or a valid message for the wrong protocol instance.

The adversary must not forge a correct process's signature or identity. Distinguish invalid forgery from a Byzantine process authenticating conflicts under its own identity.

3. Implement Algorithm 1

Implement the paper's `BUNDLE(m, sn, sender, signatures)` messages and the following state at every correct process:

- saved valid signatures, indexed by `(value, sequence_number, sender)`;
- a record of values already signed for each `(sender, sequence_number)`; and
- a record of already delivered `(sender, sequence_number)` pairs.

Use abstract signatures. For example:

```
signature = signatures.sign(signer, value, sequence_number, sender)
signatures.is_valid(signature, signer, value, sequence_number, sender)
```

Immutable Python objects or opaque tokens are enough. A valid signature must verify only for its signer and the exact tuple it covers. Correct processes sign at most one value per `(sender, sequence_number)`; Byzantine processes may sign conflicting values under their own identities. Do not implement public-key cryptography.

Follow Algorithm 1 closely and explain any small engineering decisions, such as deduplicating an identical re-broadcast.

4. Simulate and measure

For a correct sender, run a grid of at least 30 seeded trials for several valid parameter choices. Include a no-message-adversary baseline ($d = 0$) and at least two cases with $d > 0$. A useful table has columns:

n	t	d	c	Byzantine behavior	trials	delivery rate	minimum delivered	target c-d	median completion time
---	---	---	---	--------------------	--------	---------------	-------------------	------------	------------------------

Define **delivery rate** for a run as:

```
number of correct processes that mbrb-deliver the correct sender's value / c
```

Also count safety violations separately: invalid delivery, duplicate delivery, or two correct processes delivering distinct values for one sender/sequence number. A delivery rate alone cannot establish safety.

Explore fault tolerance by increasing t and d for a fixed n , including parameter sets on or beyond $n \leq 3t + 2d$. These runs are stress tests, not a disproof of the theorem: one successful schedule outside the condition does not establish correctness.

5. Compare with the paper

Compare the observations with these analytical results from Theorem 1:

- under $n > 3t + 2d$, Algorithm 1 guarantees $l = c - d$;
- with $d = 0$, it delivers to all correct processes and has a two-step bound;
- the paper gives an implementation-message bound of $2n^2$; and
- $n > 3t + 2d$ is necessary and sufficient for the abstraction in the model.

Discuss similarities and differences between the theoretical results and the simulations.

Deliverables

Submit a GitHub project containing the shared adversary component, Algorithm 1 implementation, experiment runner and raw results, one results table/plot, and a short report. The final submission may use a common adversary codebase shared with the PBFT exercise, either directly or as a clearly versioned dependency. Include instructions that reproduce your experiments from a clean checkout.